

Aproximação de π por séries: acurácia, tempo e o custo do último dígito

PREENCHER: nome do aluno · PREENCHER: professor(a)

18 de agosto de 2026

1. O que foi feito

- **Duas séries** que convergem para π , implementadas em C: **Leibniz-Gregory** (erro $\propto 1/N$) e **Nilakantha** (erro $\propto 1/4N^3$). As duas têm **custo $O(N)$ idêntico**, então a comparação isola uma única variável: a **ordem de convergência**.
- **N de 10^1 a 10^9** , medindo tempo de parede e de CPU, erro absoluto contra uma referência de 21 dígitos e o número de casas decimais corretas.
- **Mediana de 3 execuções**. Compilado com `gcc -O2 -Wall -Wextra -std=c99`, sem emitir nenhum aviso.
- **Achado principal**: existe um ponto em que processamento adicional **não compra nada** — e o limite é da **representação em ponto flutuante**, não do hardware.

2. As duas séries

Leibniz-Gregory: $\pi = 4 \cdot (1 - 1/3 + 1/5 - 1/7 + \dots)$ · termo $k = (-1)^k / (2k+1)$ · erro $\approx 1/N$

Nilakantha: $\pi = 3 + 4/(2 \cdot 3 \cdot 4) - 4/(4 \cdot 5 \cdot 6) + \dots$ · termo $k = (-1)^{k+1} \cdot 4/[a(a+1)(a+2)]$, $a = 2k$ · erro $\approx 1/4N^3$

Mesmo número de operações por termo, dentro de um fator pequeno. **A diferença não está no custo da iteração, está na qualidade da iteração.**

3. Metodologia e decisões de implementação

- **Tempo de parede** (`QueryPerformanceCounter`, nanossegundos, monotônico), porque é a única grandeza com que se calcula *speedup*. O **tempo de CPU** (`clock()`) fica em coluna separada: em código serial coincidem, no paralelo descolam.
- **Mediana, não média** — a média é deslocada por uma execução contaminada por outro processo.
- **-O2 sempre declarado**. Com `-O0` o mesmo código roda várias vezes mais devagar e os números não são comparáveis.
- **Referência de π com 21 dígitos**, não `M_PI`: `M_PI` não é padrão C, e a referência precisa de mais dígitos do que o `double` guarda, senão o erro medido seria em parte o erro da referência.
- **Sem `pow(-1.0, k)`** para alternar o sinal: seria uma chamada de função de dezenas de ciclos executada um bilhão de vezes. Multiplicar por `-1` custa 1 ciclo. Também **sem `pow(10, e)`** para gerar `N` — devolve `double` e a conversão pode dar 999999999.
- **Cronômetro só em volta do laço numérico**, com o `printf` fora: em `N = 10` a E/S dominaria a medição.

Ambiente: Intel i7-13650HX (14 núcleos: 6 P-cores + 8 E-cores, 20 threads), Windows 11, gcc 16.1.0 (MinGW-w64 UCRT).

4. Resultados

Mediana de 3 execuções. As aproximações e os erros saíram **idênticos dígito a dígito nas três** — o cálculo é determinístico, só o tempo varia. Referência: $\pi = 3,14159265358979323846$.

N	Leibniz-Gregory				Nilakantha			
	aproximação	erro	casas	t (s)	aproximação	erro	casas	t (s)
10	3,041839618929403	9,975e-02	1,00	—	3,141406718496502	1,859e-04	3,73	—
10 ²	3,131592903558554	1,000e-02	2,00	—	3,141592410971982	2,426e-07	6,62	—
10 ³	3,140592653839794	1,000e-03	3,00	—	3,141592653340544	2,492e-10	9,60	—
10 ⁴	3,141492653590034	1,000e-04	4,00	—	3,141592653589538	2,549e-13	12,59	—
10 ⁵	3,141582653589720	1,000e-05	5,00	0,0001	3,141592653589786	6,661e-15	14,18	0,0001
10 ⁶	3,141591653589774	1,000e-06	6,00	0,0009	3,141592653589787	6,217e-15	14,21	0,0010
10 ⁷	3,141592553589792	1,000e-07	7,00	0,0088	3,141592653589787	6,217e-15	14,21	0,0099
10 ⁸	3,141592643589326	1,000e-08	8,00	0,0884	3,141592653589787	6,217e-15	14,21	0,0982
10 ⁹	3,141592652588050	1,002e-09	9,00	0,8723	3,141592653589787	6,217e-15	14,21	0,9534

Tempos abaixo de 10⁵ ficam sob a granularidade útil do cronômetro.

4.1 Fator de melhoria a cada 10× em N

N	erro ÷ Leibniz	erro ÷ Nilakantha	tempo ×	N	erro ÷ Leibniz	erro ÷ Nilakantha	tempo ×
10 ²	10,0	766,4	—	10 ⁶	10,0	1,1	10,0
10 ³	10,0	973,4	—	10 ⁷	10,0	1,0	10,1
10 ⁴	10,0	977,8	9,2	10 ⁸	10,0	1,0	10,1
10 ⁵	10,0	38,3	9,9	10 ⁹	10,0	1,0	10,2

Três leituras diretas:

- Custo rigorosamente linear** — 10× iterações, 10,0–10,2× tempo, nas duas séries. Confirma O(N) e valida a medição.
- A ordem de convergência aparece nos dígitos** — Leibniz divide o erro por **exatamente 10,0** nos oito passos; Nilakantha por **~970 ≈ 10³**.
- Nilakantha para de melhorar em N = 10⁶** — o fator cai a 1,0. Não é ruído: é um teto, e ele é absoluto (Seção 5.3).

5. Análise

5.1 Retorno decrescente desde o primeiro passo

Cada casa decimal de Leibniz custa **10× a anterior**: progressão geométrica no custo para progressão aritmética no resultado. Extrapolando de N = 10⁹ (8,7 × 10⁻¹⁰ s por iteração):

casas decimais	9 (medido)	12	14	15
N necessário	10 ⁹	10 ¹²	10 ¹⁴	10 ¹⁵
tempo, 1 núcleo	0,87 s	~14,5 min	~24 h	~10 dias

E 15 casas é o limite do que um `double` comporta — não há N que vá além.

5.2 Trocar o algoritmo × paralelizar

Para **14 casas decimais corretas**:

caminho	custo	ganho
Leibniz, 1 núcleo	~24 h (N = 10 ¹⁴ , extrapolado)	—
Leibniz, 20 threads (<code>reduction</code>)	~3,6 h	6,71× medido

caminho	custo	ganho
Nilakantha, 1 núcleo	0,0001 s (N = 10 ⁵ , medido)	~10 ⁸

- *Speedup* de **6,71×** com 18 threads, **eficiência de 37%** — medido em `pi_omp.c`, no mesmo N = 10⁹ e na mesma máquina.
- A eficiência cai por duas causas: processador **heterogêneo** (P-cores rápidos e E-cores lentos desbalanceiam fatias iguais) e threads irmãs de **hyperthreading compartilhando a unidade divisora**, que é o recurso que limita este laço.
- **Karp-Flatt** estimou fração serial de **0,08 a 0,13, crescendo com p** — assinatura de overhead de paralelização, não de código serial. Amdahl com essa fração dá teto de 8× a 12×.
- **Paralelismo não corrige ordem de convergência ruim.** Vinte núcleos dividem o tempo por 6,71; seria preciso dividir por 10⁸.

5.3 O teto absoluto: 999 milhões de iterações que não mudam um bit

- De N = 10⁶ a 10⁹ o erro do Nilakantha ficou em **6,217 × 10⁻¹⁵** e a aproximação em **3,141592653589787**, sem alterar **um único dígito** em quatro ordens de grandeza.
- Causa: o termo k vale $\approx 4/(8k^3) = 0,5/k^3$. O **ulp** do `double` perto de 3,14 é $2^{-51} \approx 4,44 \times 10^{-16}$. A soma só altera algum bit se o termo passar de meio *ulp*:

$$0,5 / k^3 > 2,22 \times 10^{-16} \Rightarrow k < 1,3 \times 10^5$$

- A partir de $k \approx 130\,000$ cada termo é **absorvido** — arredondado de volta ao mesmo valor. O laço executa, gasta ~0,95 s de CPU, e o acumulador é **idêntico bit a bit**.
- É o argumento empírico contra "é só jogar mais processamento no problema": o retorno não é apenas decrescente, é **exatamente zero**. E o ponto é definido pela representação, não pelo hardware.

5.4 Efeito colateral: paralelizar muda os últimos dígitos

O erro de Leibniz em N = 10⁹ passa de $1,002 \times 10^{-9}$ com 1 thread para $9,995 \times 10^{-10}$ com 20: a soma de ponto flutuante **não é associativa**, e o número de threads muda o agrupamento dos termos. **A versão paralela não reproduz a serial bit a bit, e as duas estão corretas.** Onde há exigência de reprodutibilidade exata, isso obriga a somatórios de ordem fixa.

6. O mesmo comportamento em aplicações reais

Simulações físicas

- **Diferenças finitas de 2ª ordem, 3D:** erro $O(h^2)$. Uma casa decimal exige refinar h por $\sqrt{10} \rightarrow 32\times$ mais células e, pela **condição CFL** ($\Delta t \propto h$), 3,16× mais passos: **~100× por casa decimal**, pior que Leibniz. Daí o investimento em **métodos de ordem alta** (espectrais, Runge-Kutta 4, malha adaptativa) — a mesma troca de Leibniz por Nilakantha.
- **Monte Carlo:** erro $\propto 1/\sqrt{N}$, **100× mais amostras por casa decimal**. Justifica **redução de variância**, amostragem por importância e sequências quase-aleatórias.
- **O piso também existe:** somas de bilhões de contribuições acumulam arredondamento, e a resposta é **somatório de Kahan** ou precisão mista, não mais iterações. Em sistemas caóticos o **expoente de Lyapunov** impõe horizonte de previsibilidade de ~duas semanas na previsão do tempo, **independentemente do poder computacional**.
- **Tolerância de engenharia:** 1% num coeficiente de arrasto é atingido muito antes do limite numérico. Processamento além dela é desperdício.

Inteligência artificial

- **Leis de escala:** a perda decai como $L \propto C^{-\alpha}$ com $\alpha \approx 0,05$ nos trabalhos publicados. Com esse expoente, **reduzir a perda pela metade exige ~2²⁰ ≈ 10⁶ vezes mais compute** — mesmo formato da Seção 5.1, com

expoente pior que o de Leibniz.

- **Existe um piso:** a perda converge para um resíduo irreduzível, dado pela entropia dos dados. Nenhum compute o atravessa, como nenhuma iteração atravessa o *ulp*.
- **A saída foi trocar o algoritmo:** atenção em vez de recorrência, mistura de especialistas, destilação, quantização. Cada uma é uma "Nilakantha" — muda o expoente da curva em vez de andar mais longe sobre uma curva ruim.
- **Inversão instrutiva sobre precisão:** aqui queríamos as 15 casas do `double` ; no treinamento o caminho foi `float32` → `bfloat16` → `fp8` , jogando casas fora de propósito. **A precisão necessária é propriedade do problema, não virtude em si.**

domínio	erro × trabalho	custo de +1 dígito	piso
Leibniz	1/N	10×	precisão do <code>double</code>
Nilakantha	1/(4N ³)	2,2×	precisão do <code>double</code>
Monte Carlo	1/√N	100×	—
Dif. finitas 2ª ordem, 3D + CFL	h^2	~100×	arredondamento acumulado
Sistemas caóticos	—	—	horizonte de Lyapunov
Escala de LLMs	$C^{-0.05}$	~10 ²⁰	perda irreduzível dos dados

7. Conclusões

1. **A acurácia melhora com mais processamento, com retorno geometricamente decrescente** — custo linear em N, ganho logarítmico.
2. **A ordem de convergência domina a contagem de núcleos** — 14 casas: ~24 h em 1 núcleo, ~3,6 h em 20 threads, **0,0001 s** trocando de série.
3. **Existe um teto absoluto, e ele é da representação numérica** — 999 milhões de iterações, ~0,95 s de CPU, resultado idêntico bit a bit.
4. **Paralelizar altera os últimos dígitos**, porque a soma de ponto flutuante não é associativa.
5. **O padrão se repete em simulação física e em IA** — lei de potência no custo e um piso que o compute não atravessa.

8. Reprodução e leitura da saída

```
gcc -O2 -Wall -Wextra -std=c99 pi_serie.c -o pi_serie.exe -lm
./pi_serie.exe 6 # instantâneo ./pi_serie.exe # até 10^9, ~2 s
```

Na coluna `ERRO ABS` , as linhas do Nilakantha a partir de $N = 10^6$ repetem `6.217e-15` inalterado **enquanto T.PAREDE cresce 10× por linha** — as duas colunas lado a lado são a Seção 5.3 em uma tela.

9. Glossário dos termos empregados

termo	definição adotada, com o valor medido
Ordem de convergência	expoente com que o erro decai com o número de termos. Leibniz: 1ª ordem (1/N). Nilakantha: 3ª ordem (1/4N ³)
Erro de truncamento	erro por parar a série num termo finito. Domina em Leibniz em toda a faixa
Erro de arredondamento	erro por precisão finita das operações. Domina em Nilakantha a partir de $N \approx 10^5$
IEEE-754 dupla precisão	o <code>double</code> : 64 bits, 53 bits de mantissa , ~15–17 dígitos decimais
ulp	distância entre dois <code>double</code> consecutivos. Perto de 3,14: $2^{-51} \approx 4,44 \times 10^{-16}$
Épsilon de máquina	$2^{-52} \approx 2,22 \times 10^{-16}$, o menor ε com $1 + \varepsilon \neq 1$

termo	definição adotada, com o valor medido
Absorção / estagnação	termo pequeno demais frente ao acumulador é arredondado de volta. A partir de $k \approx 1,3 \times 10^5$ nenhum bit muda
Não associatividade	$(a+b)+c \neq a+(b+c)$; é por isso que o número de threads muda os últimos dígitos
Somatório de Kahan	mantém variável de compensação e recupera os bits perdidos; eleva o teto sem trocar o tipo
Speedup / eficiência	$t(1)/t(p)$ e $speedup/p$. Medidos: 6,71× e 37% com 18 threads
Lei de Amdahl	teto de $1/f$, com f a fração serial
Métrica de Karp-Flatt	estimativa empírica de f . Medida 0,08–0,13 , crescendo com p → overhead, não código serial
reduction	cada thread acumula em cópia privada e o compilador combina no fim — correção por privatização
Seção crítica	serializa o acesso; igualmente correta e 67× mais lenta que reduction
Condição de corrida	escrita concorrente sem sincronização. Sem reduction , π é destruído e muda a cada execução
static / dynamic / guided	políticas de distribuição das iterações; dynamic recupera parte do desbalanceamento
Núcleos heterogêneos	6 P-cores + 8 E-cores: fatias iguais deixam o P-core esperando o E-core
Hyperthreading	duas threads lógicas por núcleo físico compartilhando a unidade divisora — o gargalo deste laço
Tempo de parede × de CPU	relógio decorrido × soma do tempo dos núcleos ocupados; coincidem no serial, descolam no paralelo
Condição CFL	amarra o passo de tempo ao da malha ($\Delta t \propto h$); faz o refino custar $\sim 100\times$ por casa decimal
Expoente de Lyapunov	taxa de crescimento de perturbações em sistema caótico; impõe horizonte de previsibilidade
Leis de escala / perda irredutível	$L \propto C^{-0.05}$ e o piso dado pela entropia dos dados — o análogo do <i>ulp</i>

10. Questões metodológicas

pergunta	resposta
Por que duas séries, e não uma?	Com uma, mediria-se apenas que o erro cai. Com duas de ordens diferentes e mesmo custo $O(N)$, mede-se <i>quanto</i> cada iteração compra — isola a ordem de convergência
Por que não M_PI ?	Não é padrão C, e a referência precisa de mais dígitos que o double guarda, senão o erro medido inclui o erro da referência
Por que o fator é exatamente 10,0?	Porque o erro é $\propto 1/N$ e N foi multiplicado por 10 em cada passo. Os oito passos com 10,0 validam a medição
Por que só o Nilakantha satura?	Só ele alcança o piso : em $N = 10^5$ o erro já é da ordem do <i>ulp</i> . Leibniz, em 10^9 , ainda está em 10^{-9} — seis ordens acima do piso
E com long double ?	O teto sobe , o fenômeno permanece: é da representação, não do algoritmo
Como foi paralelizado?	#pragma omp parallel for reduction(+:soma) em pi_omp.c . Por seção crítica também é correto e custa 67× mais
Qual o <i>speedup</i> teórico?	Karp-Flatt deu fração serial 0,08–0,13 crescente com p ; Amdahl dá teto de 8× a 12× e o medido foi 6,71×
Por que a eficiência cai a 37%?	Núcleos heterogêneos desbalanceiam, e as irmãs de hyperthreading disputam a unidade divisora — que é o limite deste laço
Qual a conclusão prática?	Antes de pedir mais núcleos, verificar a ordem de convergência do método e a tolerância que o problema exige

11. Código-fonte

pi_serie.c · 229 linhas

```
1 /* =====
2 * pi_serie.c - Aproximacao de PI por series matematicas
3 *
4 * Objetivo: variar o numero de iteracoes, medir o tempo de execucao e observar
5 * como o erro em relacao ao valor real de PI diminui conforme gastamos mais
6 * processamento.
7 *
8 * Duas series sao comparadas:
9 * Leibniz-Gregory : pi = 4 * (1 - 1/3 + 1/5 - 1/7 + ...)      erro ~ 1/N
10 * Nilakantha      : pi = 3 + 4/(2*3*4) - 4/(4*5*6) + ...    erro ~ 1/(4N^3)
11 *
12 * Compilar: gcc -O2 -Wall -Wextra -std=c99 pi_serie.c -o pi_serie -lm
13 * Executar: ./pi_serie      (vai ate 10^9 iteracoes)
14 *           ./pi_serie 7    (vai ate 10^7 iteracoes)
15 * =====*/
16
17 #define _POSIX_C_SOURCE 199309L /* libera clock_gettime() no Linux/macOS */
18
19 #include <stdio.h>
20 #include <stdlib.h>
21 #include <math.h>
22 #include <time.h>
23
24 #if defined(_WIN32)
25 #include <windows.h>
26 #endif
27
28 /* Valor de referencia de PI com mais digitos do que um double consegue guardar
29 * (um double tem ~15-17 digitos decimais significativos). Nao usamos M_PI
30 * porque ele nao faz parte do C padrao - so existe como extensao. */
31 static const double PI_REF = 3.14159265358979323846;
32
33 /* Quantos valores de N vamos testar, no maximo (10^1 ate 10^9). */
34 #define MAX_EXPOENTE 9
35
36 /* -----
37 * MEDICAO DE TEMPO
38 *
39 * Existem DOIS tempos diferentes, e a diferenca entre eles e' o coracao da
40 * disciplina de programacao paralela:
41 *
42 * tempo de parede (wall time) : o relógio da parede, o que o usuario espera
43 * tempo de CPU                : soma do tempo que os nucleos ficaram ocupados
44 *
45 * Em codigo sequencial os dois sao praticamente iguais. Em codigo paralelo com
46 * 8 threads, 1 segundo de parede pode custar 8 segundos de CPU. O speedup e'
47 * sempre calculado com tempo de PAREDE.
48 * -----*/
49
50 /* Tempo de parede, em segundos, com resolucao de nanosegundos.
51 * 0 #if e' so portabilidade: Windows e POSIX tem APIs diferentes. */
52 static double tempo_parede(void)
53 {
54     #if defined(_WIN32)
55         LARGE_INTEGER freq, agora;
56         QueryPerformanceFrequency(&freq);
57         QueryPerformanceCounter(&agora);
58         return (double)agora.QuadPart / (double)freq.QuadPart;
59     #else
60         struct timespec ts;
61         clock_gettime(CLOCK_MONOTONIC, &ts);
62         return (double)ts.tv_sec + (double)ts.tv_nsec * 1e-9;
63     #endif
64 }
65
66 /* Tempo de CPU consumido pelo processo, em segundos.
67 * clock() devolve "ticks"; CLOCKS_PER_SEC converte para segundos. */
68 static double tempo_cpu(void)
69 {
70     return (double)clock() / (double)CLOCKS_PER_SEC;
71 }
72
73 /* -----
74 * AS SERIES
75 * -----*/
```

```

76
77 /* Serie de Leibniz-Gregory:
78 *
79 *    $\pi/4 = 1 - 1/3 + 1/5 - 1/7 + 1/9 - \dots$ 
80 *
81 * O termo k (começando em 0) é  $(-1)^k / (2k+1)$ .
82 *
83 * Note que NAO usamos pow(-1.0, k) para alternar o sinal: pow() é uma chamada
84 * de função cara (dezenas de ciclos) e seria executada bilhões de vezes.
85 * Multiplicar uma variável por -1 custa 1 ciclo. Trocar pow() por isso deixa o
86 * laço várias vezes mais rápido sem mudar o resultado. */
87 static double serie_leibniz(long long n)
88 {
89     double soma = 0.0;
90     double sinal = 1.0;
91     long long k;
92
93     for (k = 0; k < n; k++) {
94         soma += sinal / (double)(2 * k + 1);
95         sinal = -sinal;
96     }
97     return 4.0 * soma;
98 }
99
100 /* Serie de Nilakantha:
101 *
102 *    $\pi = 3 + 4/(2*3*4) - 4/(4*5*6) + 4/(6*7*8) - \dots$ 
103 *
104 * O termo k (começando em 1) usa  $a = 2k$  e vale  $(-1)^{k+1} * 4/(a(a+1)(a+2))$ .
105 * O denominador cresce com  $k^3$ , então o erro cai com  $1/N^3$  - muito mais rápido
106 * que Leibniz. Serve para mostrar que trocar de ALGORITMO costuma valer mais
107 * que empilhar iterações (ou núcleos) em cima de um algoritmo ruim. */
108 static double serie_nilakantha(long long n)
109 {
110     double soma = 3.0;
111     double sinal = 1.0;
112     long long k;
113
114     for (k = 1; k <= n; k++) {
115         double a = 2.0 * (double)k;
116         soma += sinal * 4.0 / (a * (a + 1.0) * (a + 2.0));
117         sinal = -sinal;
118     }
119     return soma;
120 }
121
122 /* -----
123 * APOIO
124 * ----- */
125
126 /* Quantas casas decimais estão corretas, aproximadamente.
127 * Se o erro é  $1e-5$ , então  $-\log_{10}(1e-5) = 5$  casas corretas. */
128 static double casas_corretas(double erro)
129 {
130     if (erro <= 0.0) return 17.0; /* erro zero: limite do double */
131     return -log10(erro);
132 }
133
134 /* Executa uma série para um N, mede os tempos e imprime a linha da tabela.
135 * Recebe um ponteiro para função para não duplicar todo esse código de medição
136 * uma vez para Leibniz e outra para Nilakantha. */
137 static void executa(const char *nome, double (*serie)(long long), long long n,
138                   double *erro_saida, double *tempo_saida)
139 {
140     double t0_parede, t1_parede, t0_cpu, t1_cpu;
141     double aprox, erro;
142
143     t0_parede = tempo_parede();
144     t0_cpu = tempo_cpu();
145
146     aprox = serie(n); /* <-- o trabalho de verdade */
147
148     t1_cpu = tempo_cpu();
149     t1_parede = tempo_parede();
150
151     erro = fabs(PI_REF - aprox);
152
153     printf(" %-11s %12lld %.15f %10.3e %6.2f %9.4f %9.4f\n",
154           nome, n, aprox, erro, casas_corretas(erro),
155           t1_parede - t0_parede, t1_cpu - t0_cpu);
156
157     *erro_saida = erro;
158     *tempo_saida = t1_parede - t0_parede;
159 }

```



```

160
161 /* -----
162  * PROGRAMA PRINCIPAL
163  * -----*/
164 int main(int argc, char *argv[])
165 {
166     int expoente_max = MAX_EXPOENTE;
167     int e, i;
168
169     /* Arrays para guardar os resultados e analisar a convergencia no final. */
170     long long ns[MAX_EXPOENTE + 1];
171     double erro_leib[MAX_EXPOENTE + 1], tempo_leib[MAX_EXPOENTE + 1];
172     double erro_nila[MAX_EXPOENTE + 1], tempo_nila[MAX_EXPOENTE + 1];
173     int qtd = 0;
174
175     /* argv[1] opcional: expoente maximo, para nao esperar muito em teste. */
176     if (argc > 1) {
177         expoente_max = atoi(argv[1]);
178         if (expoente_max < 1 || expoente_max > MAX_EXPOENTE) {
179             fprintf(stderr, "Uso: %s [expoente entre 1 e %d]\n",
180                     argv[0], MAX_EXPOENTE);
181             return 1;
182         }
183     }
184
185     printf("Valor de referencia de PI : %.15f\n", PI_REF);
186     printf("Precisao do tipo double : ~15-17 digitos decimais\n\n");
187
188     printf(" %-11s %12s %-17s %10s %6s %9s %9s\n",
189            "SERIE", "N", "APROXIMACAO", "ERRO ABS", "CASAS", "T.PAREDE", "T.CPU");
190     printf(" -----
191            "-----\n");
192
193     for (e = 1; e <= expoente_max; e++) {
194         long long n = 1;
195         int j;
196         for (j = 0; j < e; j++) n *= 10; /* n = 10^e sem usar pow() */
197
198         ns[qtd] = n;
199         executa("Leibniz", serie_leibniz, n, &erro_leib[qtd], &tempo_leib[qtd]);
200         executa("Nilakantha", serie_nilakantha, n, &erro_nila[qtd], &tempo_nila[qtd]);
201         printf("\n");
202         qtd++;
203     }
204
205     /* ---- Analise da convergencia -----
206     * A cada passo multiplicamos N por 10. Se o erro cai por um fator ~10, a
207     * convergencia e' de 1a ordem (erro ~ 1/N). Se cai por ~1000, e' de 3a
208     * ordem (erro ~ 1/N^3). O tempo, em ambos os casos, cresce ~10x, porque o
209     * custo e' O(N) nas duas series. */
210     printf("Fator de reducao do erro e de aumento do tempo a cada 10x em N:\n\n");
211     printf(" %14s %12s %12s %12s\n",
212            "N", "ERRO/10x LEIB", "ERRO/10x NILA", "TEMPO/10x");
213     printf(" -----
214            "-----\n");
215     for (i = 1; i < qtd; i++) {
216         double fator_leib = erro_leib[i - 1] / erro_leib[i];
217         double fator_nila = erro_nila[i - 1] / erro_nila[i];
218         double fator_tempo = (tempo_leib[i - 1] > 0.0)
219                             ? tempo_leib[i] / tempo_leib[i - 1] : 0.0;
220         printf(" %14lld %12.1f %12.1f %12.1f\n",
221                ns[i], fator_leib, fator_nila, fator_tempo);
222     }
223
224     printf("\nLeitura: com Leibniz, 10x mais trabalho compra ~1 casa decimal.\n");
225     printf("Com Nilakantha, o mesmo trabalho compra ~3 casas - ate a serie\n");
226     printf("bater no limite de precisao do double (~1e-16), onde mais\n");
227     printf("iteracoes so acumulam erro de arredondamento e nao melhoram nada.\n");
228
229     return 0;
230 }

```